

Presented by:

Achi Lara, Beșliu Beatrice, Antonescu Ema
(CTI-RO C1.1)

8-bit ALU Simulation

Obiectivele proiectului

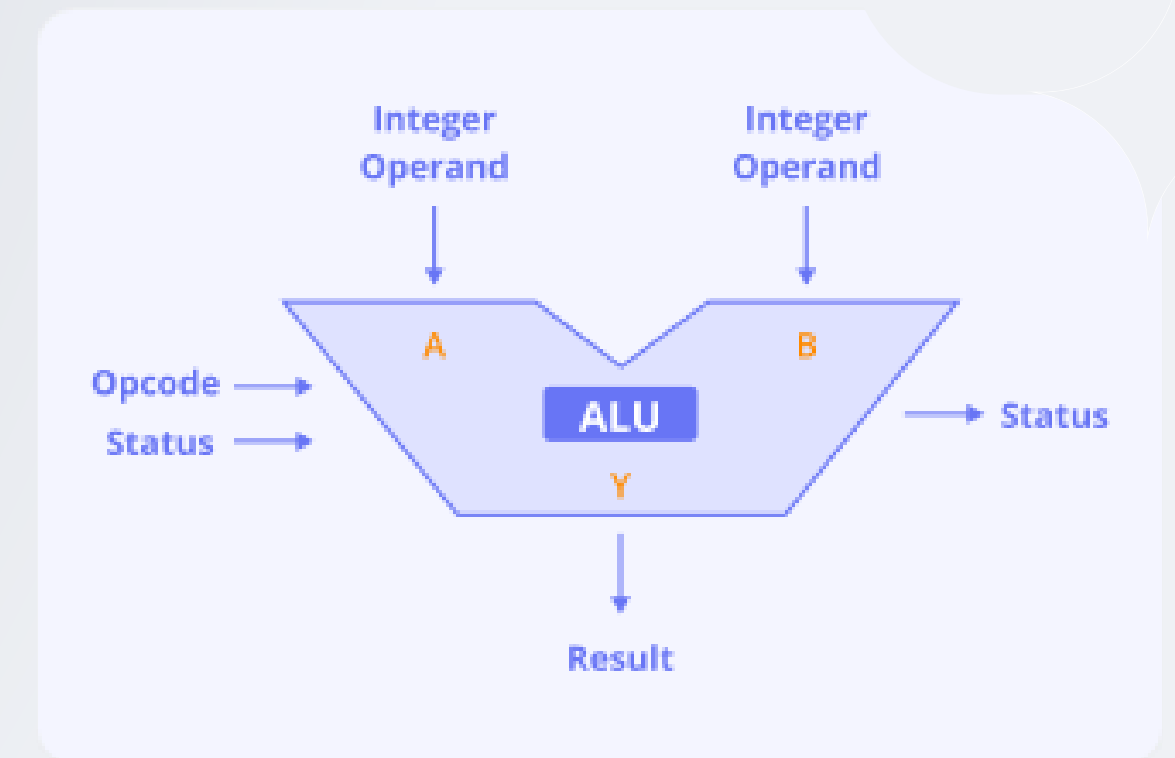
Acest proiect constă în proiectarea și simularea unei Unități Aritmetice și Logice (ALU) pe 8 biți, utilizând limbajul Verilog HDL. Scopul principal este realizarea unei arhitecturi structurale, în care fiecare operație aritmetică este implementată prin module hardware distincte, interconectate logic.

ALU-ul proiectat suportă patru operații fundamentale: adunare, scădere, înmulțire și împărțire. Selectarea operației se realizează prin intermediul unui semnal de control (opcode), care determină rezultatul final transmis la ieșire.

Obiectivul proiectului este de a evidenția modul de implementare hardware al operațiilor aritmetice, precum și integrarea acestora într-un sistem coerent, testabil și extensibil.

Ce este un ALU?

Un ALU (Arithmetic Logic Unit), sau Unitatea Aritmetică și Logică, este o componentă esențială a procesorului care se ocupă de executarea operațiilor aritmetice și logice asupra datelor binare. Aceasta primește date binare și un semnal de control care stabilește operația ce trebuie realizată. ALU poate efectua calcule precum adunarea, scăderea, înmulțirea și împărțirea, dar și operații logice precum AND, OR, XOR și NOT. Rezultatul obținut este trimis înapoi către procesor pentru a fi folosit în execuția instrucțiunilor, contribuind astfel atât la efectuarea calculelor, cât și la luarea deciziilor logice în sistemele digitale.



Decizie de proiectare - arhitectură modulară

Pentru realizarea ALU-ului, am ales o arhitectură modulară, în care fiecare operație este implementată într-un bloc separat. Această organizare împarte sistemul în componente distincte pentru adunare, scădere, înmulțire și împărțire, ceea ce face designul mai clar și mai ușor de urmărit. Fiecare modul poate fi dezvoltat și testat individual, fără a afecta celelalte părți ale sistemului. În plus, această abordare permite optimizarea separată a fiecărei operații, de exemplu utilizarea algoritmului Booth pentru înmulțire și SRT pentru împărțire. Rezultatul final este selectat în funcție de semnalul de control (opcode), ceea ce asigură flexibilitate în funcționarea ALU-ului.

Adunare

Operația de adunare este realizată în cadrul modului adder8, folosind o arhitectură de tip Ripple Carry Adder pe 8 biți. Aceasta este construită prin interconectarea a opt module elementare de tip full adder, fiecare responsabil pentru calculul sumei unui bit corespunzător din operanzii de intrare.

Fiecare full adder primește trei semnale de intrare: bitul curent din operandul A, bitul corespunzător din operandul B și semnalul de transport (carry-in) provenit de la etapa anterioară. Acesta generează două ieșiri: suma parțială (sum) și semnalul de transport (carry-out).

Transportul este propagat secvențial de la bitul cel mai puțin semnificativ (LSB) către cel mai semnificativ (MSB), ceea ce definește comportamentul de tip „ripple”. Primul full adder utilizează un carry-in fix (0), iar fiecare etapă ulterioară folosește carry-ul generat anterior.

Rezultatul final este vectorul S[7:0], care reprezintă suma celor doi operanzi, iar Cout indică transportul final (posibil overflow în cazul numerelor nesemnate).

Messages			
+ ◆	/tb_alu/A	00001010	00001010
+ ◆	/tb_alu/B	00000101	00000101
+ ◆	/tb_alu/opcode	00	00
+ ◆	/tb_alu/RESULT	0000000000001111	0000000000001111
+ ◆	/tb_alu/REMAINDER	00000000	00000000

Scădere

Operația de scădere este implementată în modulul sub8 folosind metoda complementului față de 2. În loc să se construiască un circuit separat de scădere, se reutilizează modulul de adunare (adder8), transformând scăderea într-o operație de adunare.

Mai exact, operandul B este transformat în complement față de 2 prin inversarea tuturor biților ($\sim B$) și adăugarea valorii 1. Acest rezultat (B2) reprezintă valoarea $-B$ în reprezentare binară.

Ulterior, se efectuează adunarea dintre A și B2 folosind același adder pe 8 biți. Rezultatul obținut (D) reprezintă diferența $A - B$.

Această abordare permite o implementare eficientă, reutilizând hardware-ul existent pentru adunare și evitând necesitatea unui circuit dedicat pentru scădere.

Messages			
+ ◆	/tb_alu/A	00010100	00010100
+ ◆	/tb_alu/B	00000111	00000111
+ ◆	/tb_alu/opcode	01	01
+ ◆	/tb_alu/RESULT	0000000000001101	0000000000001101
+ ◆	/tb_alu/REMAINDER	00000000	00000000

Înmulțire

Înmulțirea este realizată în modulul booth_radix4 folosind algoritmul Booth Radix-4, care procesează câte 2 biți ai multiplicatorului pe iterație. Pentru fiecare grup de 3 biți se decide dacă se adună, se scade sau nu se modifică multiplicandul (M), deplasat corespunzător la stânga.

Rezultatul parțial este acumulat în variabila temp, iar la final se obține produsul pe 16 biți (P). Această metodă reduce numărul de operații față de înmulțirea clasică.

Messages			
+ ◆	/tb_alu/A	00000110	00000110
+ ◆	/tb_alu/B	00000100	00000100
+ ◆	/tb_alu/opcode	10	10
+ ◆	/tb_alu/RESULT	0000000000011000	0000000000011000
+ ◆	/tb_alu/REMAINDER	00000000	00000000

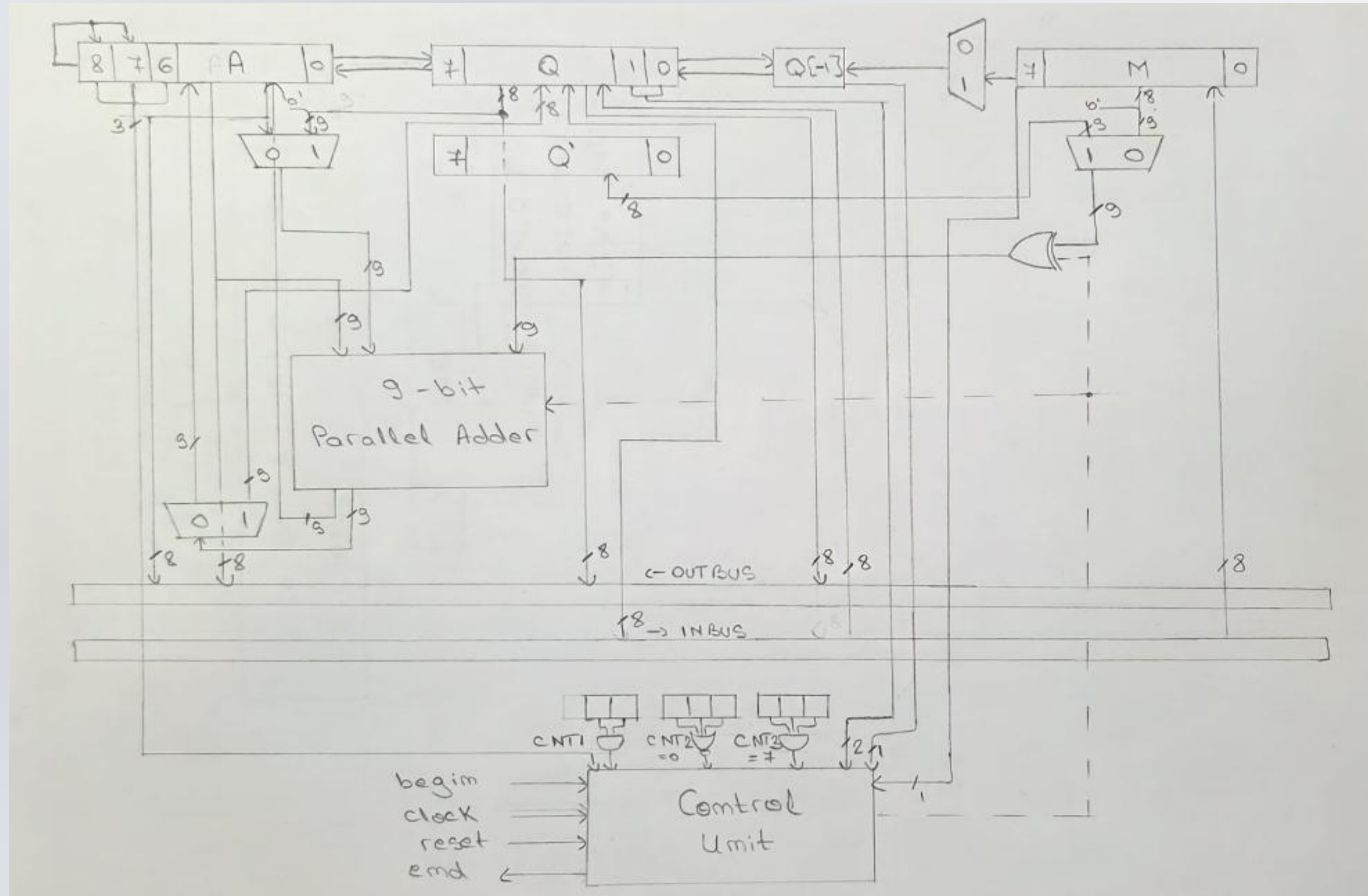
Împărțire

Împărțirea este implementată în modulul srt2_div printr-un algoritm iterativ de tip Radix-2. La fiecare pas, restul este deplasat la stânga și se adaugă câte un bit din deîmpărțit.

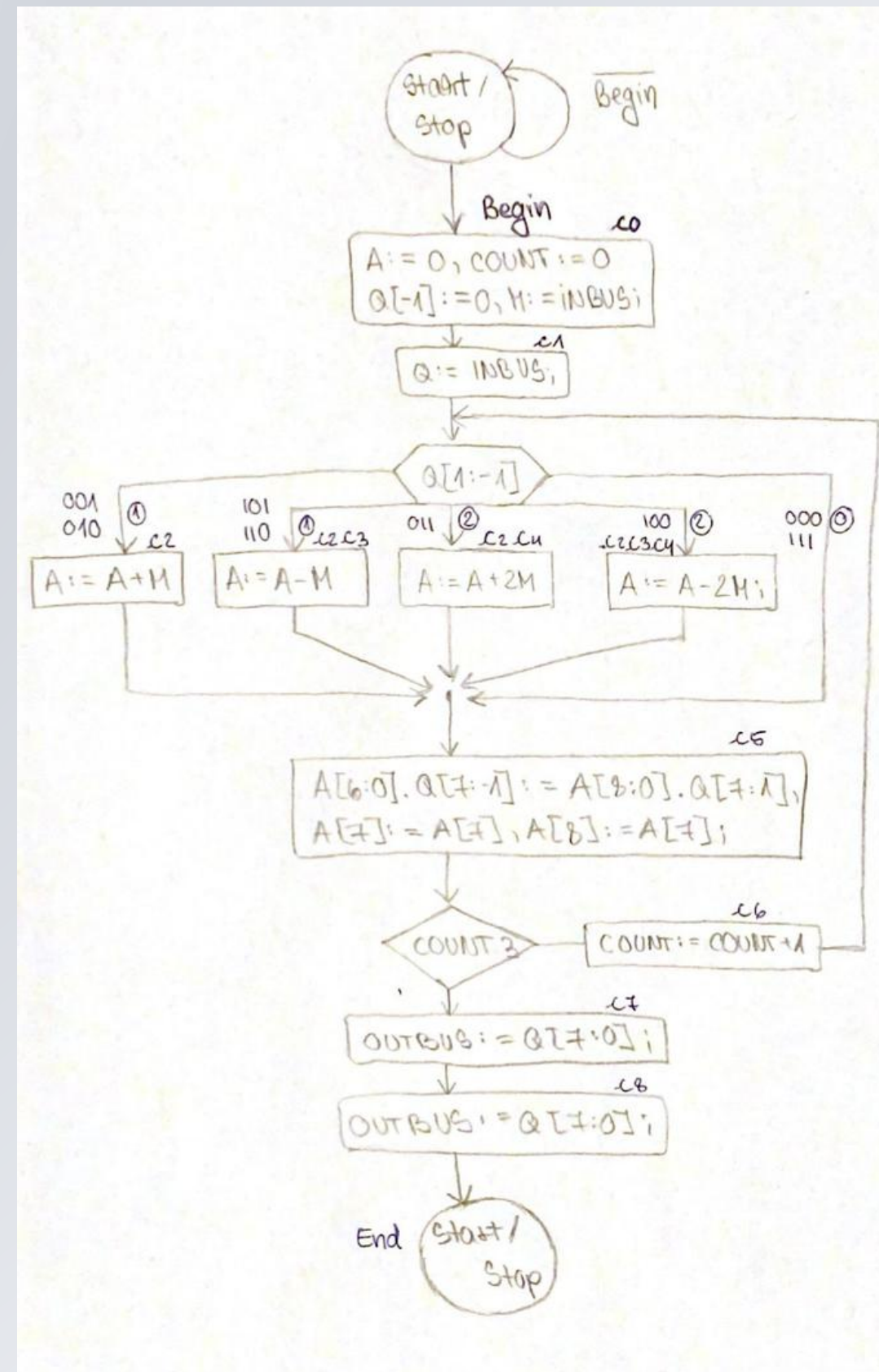
Dacă valoarea obținută este mai mare sau egală decât împărțitorul, se face scăderea și se setează bitul corespunzător din cât la 1; altfel rămâne 0. La final se obțin câtul (Quotient) și restul (Remainder). Împărțirea la zero este tratată separat.

Messages			
+ ◆	/tb_alu/A	00010100	00010100
+ ◆	/tb_alu/B	00000011	00000011
+ ◆	/tb_alu/opcode	11	11
+ ◆	/tb_alu/RESULT	000000000000110	000000000000110
+ ◆	/tb_alu/REMAINDER	00000010	00000010

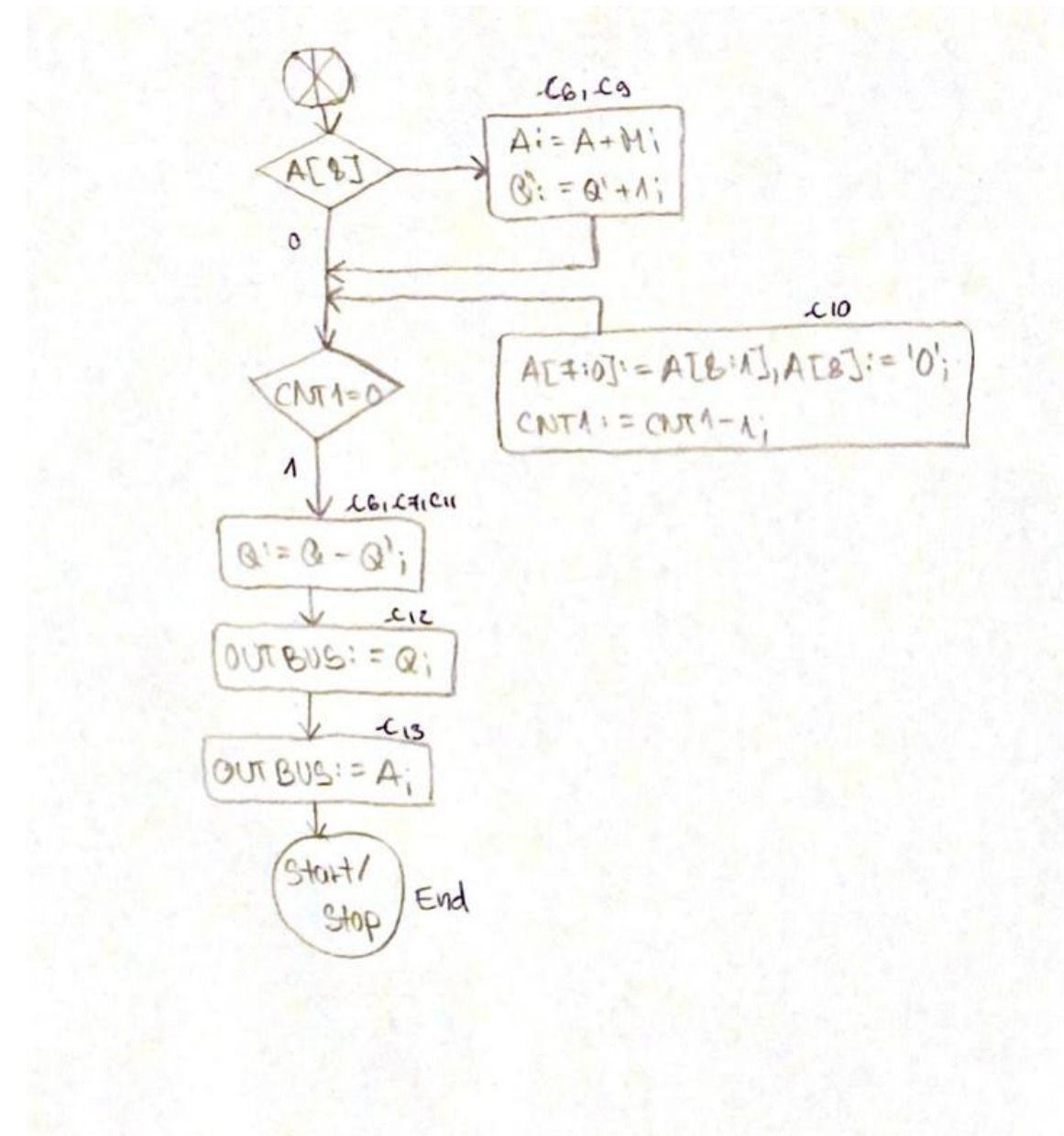
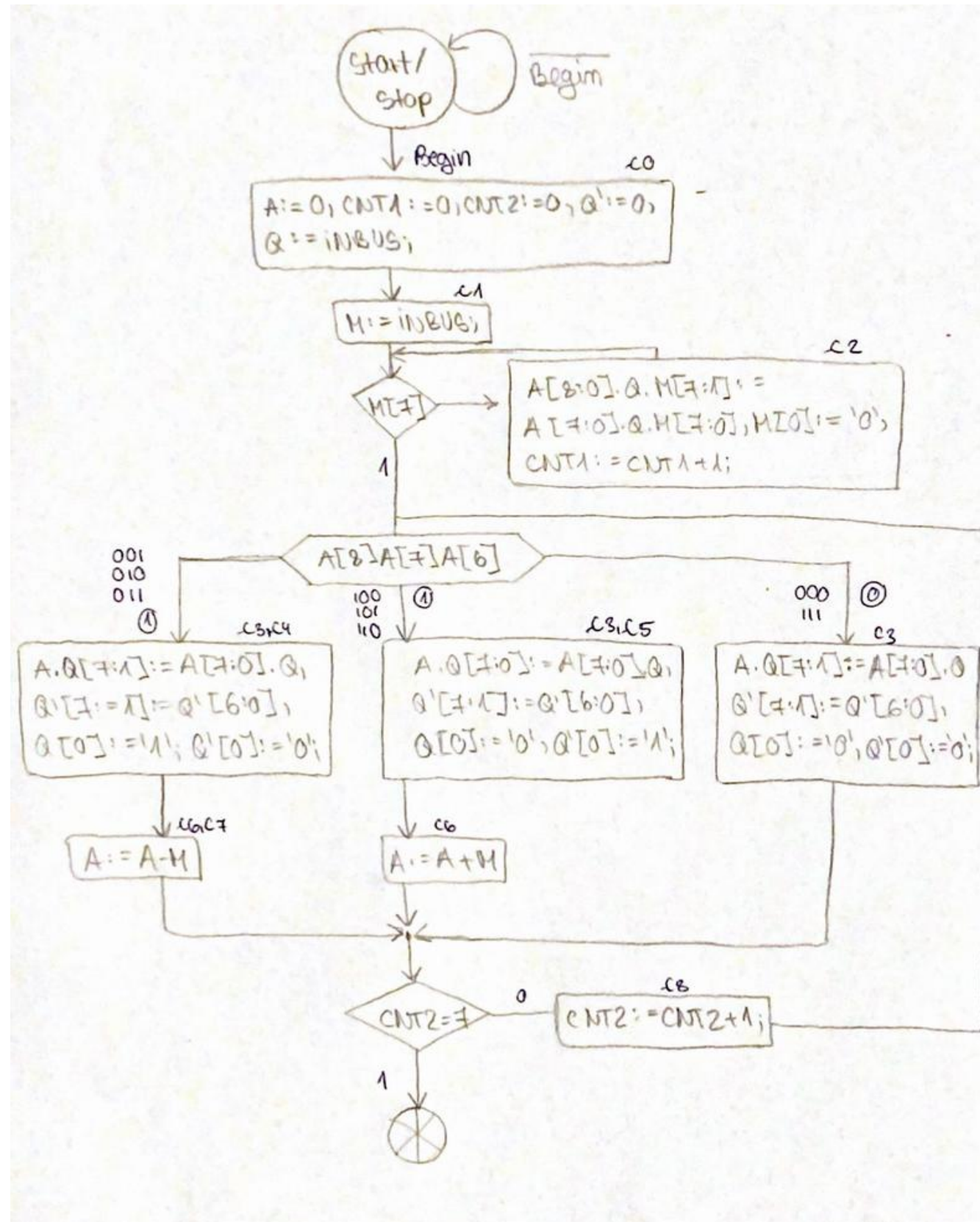
Diagrama Hardware



Booth Radix-4



SRT Radix-2



Modul ALU

```
module alu_top(  
    input [7:0] A,  
    input [7:0] B,  
    input [1:0] opcode,  
  
    output reg [15:0] RESULT,  
    output reg [7:0] REMAINDER  
);  
  
    wire [7:0] add_raw;  
    wire [7:0] sub_raw;  
    wire [15:0] mul_raw;  
    wire [7:0] q_raw;  
    wire [7:0] r_raw;  
  
    wire c;  
  
    adder8      U1(A,B,add_raw,c);  
    sub8        U2(A,B,sub_raw);  
    booth_radix4 U3(A,B,mul_raw);  
    srt2_div    U4(A,B,q_raw,r_raw);  
  
    always @(*) begin  
  
        RESULT    = 16'b0;  
        REMAINDER = 8'b0;  
  
        case(opcode)
```

```
        case(opcode)  
  
            // ADD  
            2'b00: begin  
                RESULT = {8'b0, add_raw};  
            end  
  
            // SUB  
            2'b01: begin  
                RESULT = {8'b0, sub_raw};  
            end  
  
            // MUL  
            2'b10: begin  
                RESULT = mul_raw;  
            end  
  
            // DIV  
            2'b11: begin  
                RESULT    = {8'b0, q_raw};  
                REMAINDER = r_raw;  
            end  
  
        endcase  
  
    end  
  
endmodule
```


TESTBENCH

```
module tb_alu;

    reg [7:0] A,B;
    reg [1:0] opcode;

    wire [15:0] RESULT;
    wire [7:0] REMAINDER;

    alu_top DUT(
        A,B,opcode,
        RESULT,
        REMAINDER
    );

    initial begin

        $display("==== ALU CLEAN TEST =====");

        // ADD
        //A=10; B=5; opcode=2'b00; #10;

        // SUB
        //A=20; B=7; opcode=2'b01; #10;

        // MUL
        //A=6; B=4; opcode=2'b10; #10;

        // DIV
        A=20; B=3; opcode=2'b11; #10;

        // DIV
```

```
        initial begin

            $display("==== ALU CLEAN TEST =====");

            // ADD
            //A=10; B=5; opcode=2'b00; #10;

            // SUB
            //A=20; B=7; opcode=2'b01; #10;

            // MUL
            //A=6; B=4; opcode=2'b10; #10;

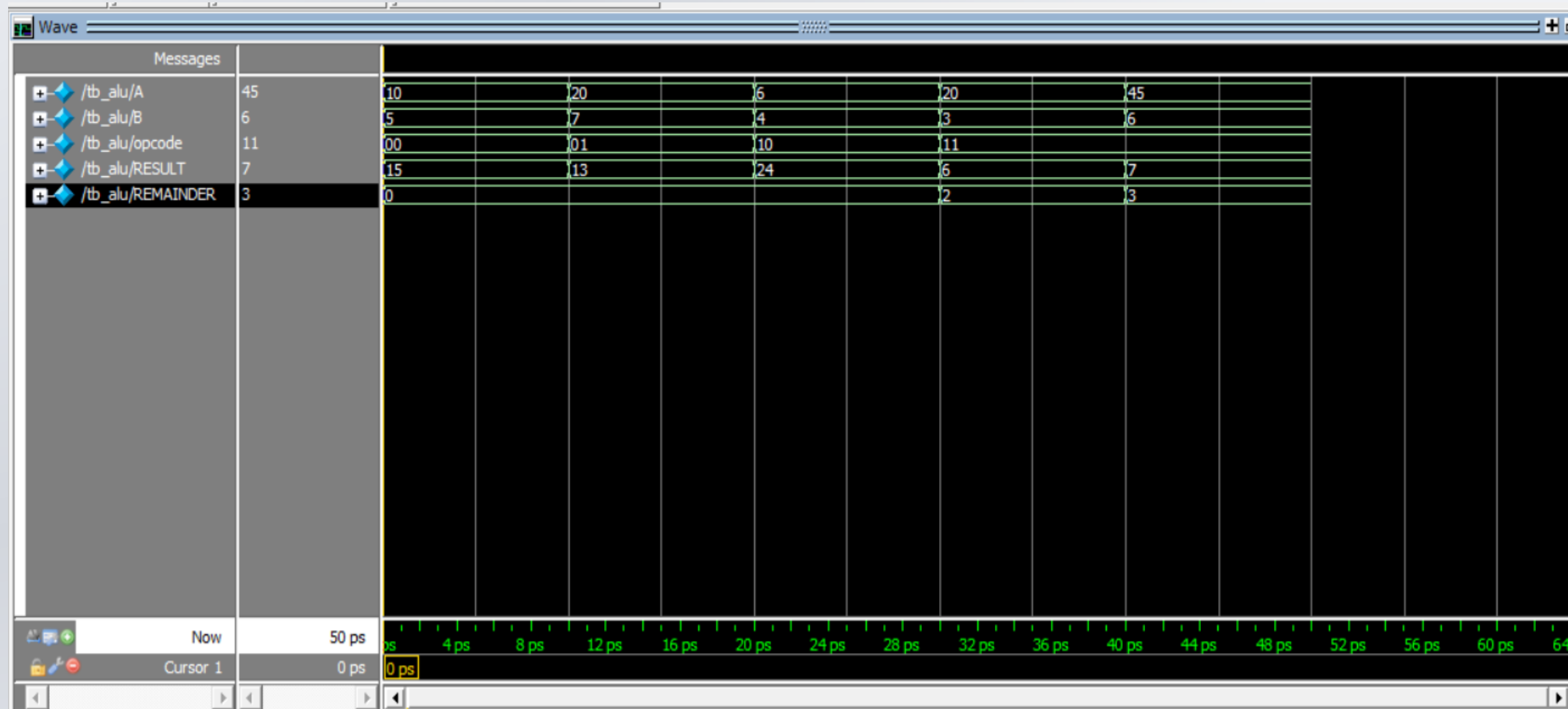
            // DIV
            A=20; B=3; opcode=2'b11; #10;

            // DIV
            //A=45; B=6; opcode=2'b11; #10;

            $stop;

        end

    endmodule
```



Concluzie

În urma simulărilor realizate, ALU-ul proiectat a demonstrat funcționarea corectă pentru toate operațiile implementate: adunare, scădere, înmulțire și împărțire. Rezultatele obținute în testbench au fost conforme cu valorile teoretice, confirmând corectitudinea fiecărui modul (adder, subtractor, multiplier și divider). Semnalul opcode a selectat corespunzător operația dorită, validând integrarea corectă la nivelul modulului alu_top.

O provocare importantă a fost diferența de dimensiune a rezultatelor: adunarea și scăderea produc rezultate pe 8 biți, în timp ce înmulțirea generează 16 biți. Aceasta a fost rezolvată prin extinderea rezultatelor pe 8 biți la 16 biți prin concatenare, asigurând o interfață unitară pentru ieșirea finală.

De asemenea, a fost necesară tratarea cazului de împărțire la zero în modulul de divizare, unde s-a introdus o condiție explicită pentru a evita rezultate nedeterminate, setând ieșirile la o valoare prestabilită.

În concluzie, designul ALU este stabil și funcțional, iar problemele întâmpinate au fost legate în principal de compatibilitatea semnalelor și de integrarea modulelor, fiind rezolvate eficient.



Thank You!